

# Hibernate — Quick Revision Notes

## Java Objects vs Relational Tables

Java uses classes/objects, but databases use tables/rows. They are structurally different — this mismatch is called the **object-relational impedance mismatch**.

## Spring JDBC Persistence

With Spring JDBC, you still write SQL manually and pass values yourself. It's cleaner than raw JDBC but doesn't understand the Student ↔ students table relationship.

## Manual SQL Writing

For every operation (insert, select, update, delete) you must write SQL by hand. Fine for small apps, but painful when you have hundreds of entities.

## Manual Parameter Binding

You must pass values in the exact (?) order in the SQL query. A wrong order won't give a compile error — just a runtime bug.

## Manual Row-to-Object Mapping

When reading data, you must manually convert each DB row into a Java object using a `RowMapper`.

## Object-Relational Mapping (ORM)

ORM automatically maps Java classes → tables, objects → rows, fields → columns, references → foreign keys, and collections → related rows.

## Responsibilities of an ORM

An ORM framework handles: mapping metadata, SQL generation, parameter binding, result-set mapping, relationship management, caching, and transaction integration — so you don't do it manually.

## What is Hibernate?

Hibernate is an ORM framework for Java that sits on top of JDBC (Application → Hibernate → JDBC → Driver → Database).

## JPA / Jakarta Persistence

JPA (now called Jakarta Persistence) is just a **specification/interface** (like `Entity`, `Id`, `EntityManager`). It defines rules but doesn't do the actual work.

## Hibernate as a JPA Provider

Hibernate is the **implementation** of the JPA specification — just like `ArrayList` implements the `List` interface. JPA = the contract, Hibernate = who fulfills it.

## Setting Up Hibernate with Spring Boot

Need dependencies: Spring Data JPA (for config) + MySQL Driver. Configure DB connection in `application.properties` (url, username, password).

## ddl-auto Options (Schema Generation)

- **none** → Hibernate does nothing to schema.
- **validate** → Just checks schema matches entities, doesn't change anything.
- **update** → Updates schema to match entities (good for learning, not for production).
- **create** → Drops and recreates schema every startup (data lost).
- **create-drop** → Creates schema on startup, drops it when app closes.

| In real production, use Flyway/Liquibase instead of `update`.

## SQL Logging

`spring.jpa.show-sql=true` shows generated SQL in console; `format_sql=true` makes it readable.

## Entity Class (@Entity)

A Java class marked `@Entity` represents a database table. `@Table(name=...)` maps it to a specific table name. `@Id` marks the primary key, `@GeneratedValue` auto-generates it.

## Repository & EntityManager

`EntityManager` is the core JPA interface used to talk to the persistence provider (Hibernate) — used for `persist()`, `find()`, `remove()`, etc.

### Create (persist())

`entityManager.persist(student)` saves a new entity into the database.

### Read (find())

`entityManager.find(Student.class, id)` fetches an entity by its ID.

## Update (Dirty Checking)

Just change a field on a **managed entity** (`student.setName(...)`) — no need to call save/update. Hibernate automatically detects the change and updates DB on commit. This magic is called **dirty checking**.

## Delete (remove())

`entityManager.remove(student)` marks entity for deletion; actual DB delete happens during flush.

## @Column Annotation

Customizes how a field maps to a column: `name`, `nullable`, `unique`, `length`, `insertable`, `updatable`.

## BigDecimal (precision & scale)

Used for decimal numbers. `precision` = total digits, `scale` = digits after decimal point. E.g., 98.75 → precision=4, scale=2.

## Date/Time Mapping

- `LocalDate` → DATE (just date)
- `LocalTime` → TIME (just time)
- `LocalDateTime` → TIMESTAMP (date + time)

## Enum Mapping

`@Enumerated(EnumType.STRING)` stores enum values as readable text (e.g., ACTIVE) in DB instead of numbers.

## Large Objects (@Lob)

Use `@Lob` for large text or binary data (like images). In production, usually store files externally (object storage) and just save the URL in DB.

## @Transient

Marks a field that should NOT be saved to database — Hibernate simply ignores it.

## Attribute Converters (@Convert)

Used when Java type and DB type don't match naturally (e.g., DB stores "Y"/"N" but Java wants Boolean). You write a custom converter class implementing `AttributeConverter`.

## Entity vs Value Type

- Entity** = has its own independent identity (e.g., Student, Department, Course).
- Value type** = has no identity of its own, exists only as part of another object (e.g., Address).

## @Embeddable & @Embedded

`@Embeddable` marks a class as a value type (no own @Id). `@Embedded` is used inside an entity to include that value type — its fields get flattened into the same table.

## **@AttributeOverride**

Used when you have two embedded objects of the same type (e.g., `currentAddress` & `permanentAddress`) — renames columns to avoid conflicts.

## **@ElementCollection**

Used to store a collection (Set/List) of simple values or embeddables that belong to one entity (like skills or phone numbers) — stored in a separate table, but not as independent entities.

## **Entity Association vs Embedded Value (When to Use What)**

- Use **Entity** when object has independent identity, can exist on its own, shared/queried/updated separately (e.g., `Department`, `Course`, `Teacher`).
- Use **Embeddable/Value type** when object has no identity of its own and always belongs to one owner (e.g., `Address`).